```
export async function POST(req) {
       const body = await req.json();
  const input = body.input || "";
  const humanMessage = new HumanMessage(input);
  const { client, tools } =
await createMCPClient();
  const model = new ChatGoogleGenerativeAI({
    model: "gemini-2.5-flash",
    apiKey: process.env.GEMINI_API_KEY,
  });
  const agent =
createReactAgent({ llm: model, tools });
  const result = await agent.invoke({ messages: [humanMessage] });
  await client.close();

  // Stream result back in UIMessage format
  return
createUIMessageStreamResponse({stream: result});
}
```

Initializes the MCP client and tools for the agent

Configures a LangChain React chain agent with Gemini and the MCP tools

Invokes the agent, closes the client, and streams the result back to the frontend

> **NOTE** This listing is simplified to reduce size. The full version includes helper functions for normalizing user messages, handling streaming, and writing structured UI events.

The listing implements the four steps outlined previously. It starts with `createMCPClient()`, which establishes a connection to the Chuck Norris MCP server via `stdio` and fetches the tools it provides. In the `POST` handler, the user's input is wrapped as a `HumanMessage`. The Gemini model is initialized, and a LangChain React agent is created by combining the model with the MCP tools. When the agent is invoked with the message, it decides whether to use the `get-chuck-joke` tool. After the agent produces its output, the MCP client is closed, and the result is streamed back in a UI-friendly format to the frontend.

THE FRONTEND

On the frontend, the application uses the `useChat` hook (from the Vercel AI SDK) to manage chat state and send user input to our `/api/chat` endpoint. Messages returned from the backend come back as structured UI events. For example, when the agent calls the Chuck Norris MCP tool, the response is rendered with a custom `ChuckNorrisFactCard` component. This gives the chat a richer feel than plain text and highlights tool outputs in a user-friendly way.

The rest of the frontend is standard Next.js with React: an input box for the user, a chat history display, and conditional rendering of special cards when the backend streams structured tool responses. The important part is that we don't need extra glue code—the backend already formats responses as UI events, and the frontend simply listens and renders. Instead of showing more code here, I give you a feel for the result by showing the application live in figure 12.8.